

Tp4

(Bouyahi Mohamed)

1) Les objectifs:

Comprendre **les namespaces** (espaces de noms) : ils isolent certaines ressources (processus, réseau, système de fichiers, etc.).

Comprendre **les cgroups** (control groups) : ils limitent et surveillent l'utilisation des ressources (CPU, mémoire...).

Construire un conteneur "à la main", sans Docker, en utilisant uniquement les outils du noyau Linux.

2) Le travail

Etape 1

- On a écrit hostname et on a affiché le nom de host c'est osboxes

- on a vérifié les IDs des namespaces associés à un processus Linux.

```
osboxes@osboxes:~$ hostname
osboxes
osboxes@osboxes:~$ lsns -p $$
      NS TYPE   NPROCS   PID USER   COMMAND
4026531834 time      72    1680 osboxes /usr/bin/pipewire
4026531835 cgroup    72    1680 osboxes /usr/bin/pipewire
4026531836 pid      72    1680 osboxes /usr/bin/pipewire
4026531837 user     72    1680 osboxes /usr/bin/pipewire
4026531838 uts      72    1680 osboxes /usr/bin/pipewire
4026531839 ipc      72    1680 osboxes /usr/bin/pipewire
4026531840 net      72    1680 osboxes /usr/bin/pipewire
4026531841 mnt      70    1680 osboxes /usr/bin/pipewire
osboxes@osboxes:~$
```

Etape 2: Créer un espace de noms UTS

On a créé un autre espace avec un NouveauNom et on va maintenant faire la comparaison

```
osboxes@osboxes:~$ hostname
osboxes
osboxes@osboxes:~$ lsns -p $$
      NS TYPE  NPROCS  PID USER  COMMAND
4026531834 time      72   1680 osboxes /usr/bin/pipewire
4026531835 cgroup     72   1680 osboxes /usr/bin/pipewire
4026531836 pid       72   1680 osboxes /usr/bin/pipewire
4026531837 user      72   1680 osboxes /usr/bin/pipewire
4026531838 uts       72   1680 osboxes /usr/bin/pipewire
4026531839 ipc       72   1680 osboxes /usr/bin/pipewire
4026531840 net       72   1680 osboxes /usr/bin/pipewire
4026531841 mnt       70   1680 osboxes /usr/bin/pipewire
osboxes@osboxes:~$
```

```
root@osboxes: /home/osboxes
osboxes@osboxes: ~ x root@osboxes: /home/osboxes x
osboxes@osboxes:~$ sudo unshare --uts bash
[sudo] password for osboxes:
root@osboxes:/home/osboxes# hostname NouveauNom
root@osboxes:/home/osboxes# hostname
NouveauNom
root@osboxes:/home/osboxes# lsns -p $$
      NS TYPE  NPROCS  PID USER  COMMAND
4026531834 time     189     1 root /sbin/init splash
4026531835 cgroup    189     1 root /sbin/init splash
4026531836 pid       189     1 root /sbin/init splash
4026531837 user      187     1 root /sbin/init splash
4026531839 ipc       189     1 root /sbin/init splash
4026531840 net       187     1 root /sbin/init splash
4026531841 mnt       172     1 root /sbin/init splash
4026532216 uts        2  9085 root bash
root@osboxes:/home/osboxes#
```

Etape3:comparer avec le nom d hote

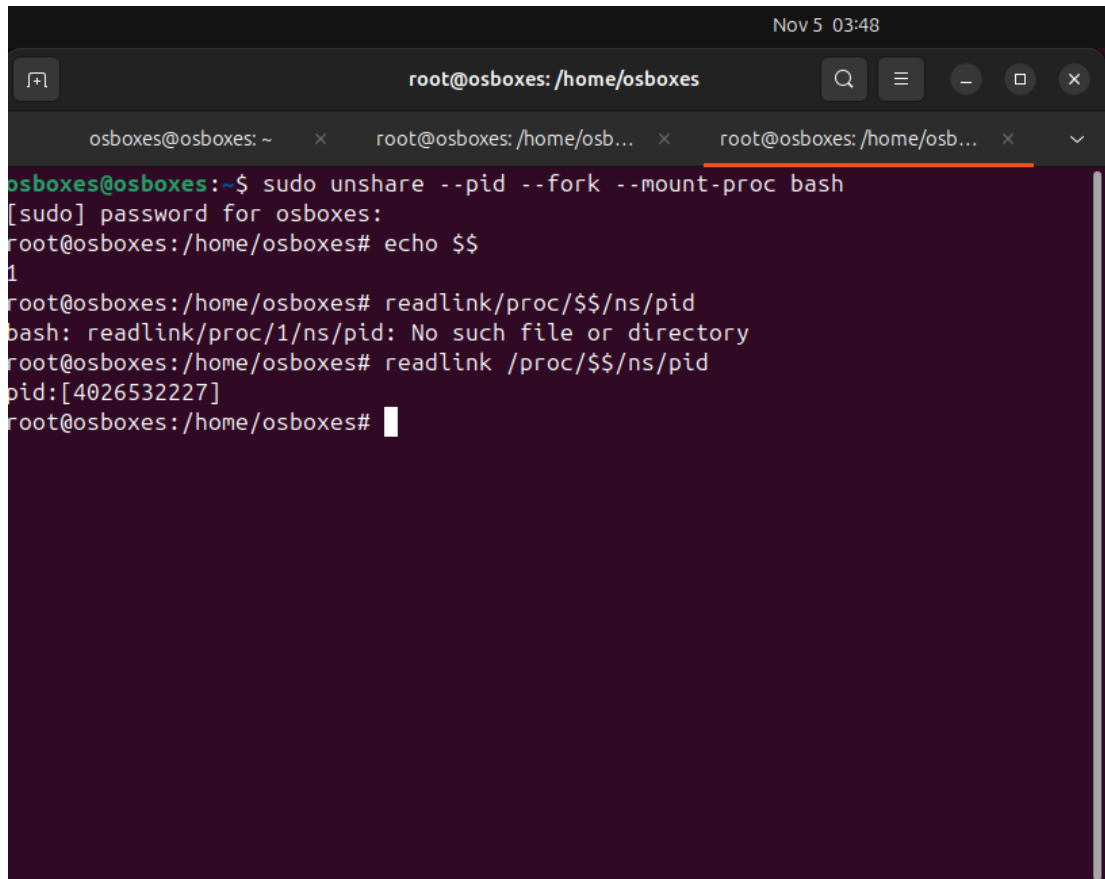
Le nom seulement change dans le deuxieme terminal mais le premier terminal reste le meme

L'espace de noms UTS permet à chaque conteneur d'avoir son propre nom d'ordinateur sans changer celui de la machine réelle.

Exercice 2

Etape1:

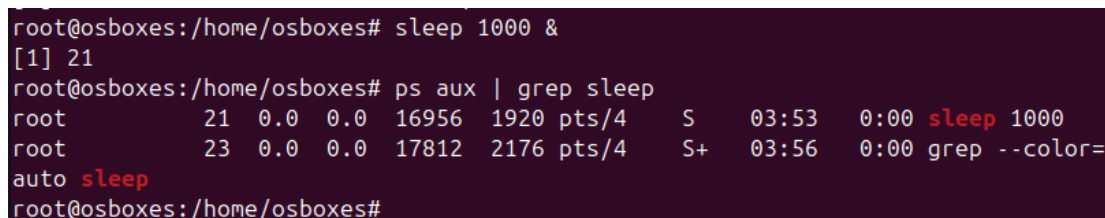
On va creer un nouveau processus bash dans un nouvel espace de noms PID en utilisant la commande `sudo unshare --pid --fork --mount-proc bash`
Puis on va verifier notre Pid par la commande `echo $$` et on a trouver 1 comme Pid
Et on va identifier l ID de l' espace de nom par `readlink`



```
Nov 5 03:48
root@osboxes: /home/osboxes
osboxes@osboxes: ~ x root@osboxes: /home/osb... x root@osboxes: /home/osb... x
osboxes@osboxes:~$ sudo unshare --pid --fork --mount-proc bash
[sudo] password for osboxes:
root@osboxes:/home/osboxes# echo $$
1
root@osboxes:/home/osboxes# readlink/proc/$$/ns/pid
bash: readlink/proc/1/ns/pid: No such file or directory
root@osboxes:/home/osboxes# readlink /proc/$$/ns/pid
pid:[4026532227]
root@osboxes:/home/osboxes#
```

Étape 2 : Démarrer un processus dans l'espace de noms

On va lancer nouveau processus en arrière-plan (par exemple, `sleep 1000`) et observez l'ID de ce processus dans l'espace de noms : `sleep 1000 &`



```
root@osboxes:/home/osboxes# sleep 1000 &
[1] 21
root@osboxes:/home/osboxes# ps aux | grep sleep
root      21  0.0  0.0 16956 1920 pts/4    S   03:53   0:00 sleep 1000
root      23  0.0  0.0 17812 2176 pts/4    S+  03:56   0:00 grep --color=
auto sleep
root@osboxes:/home/osboxes#
```

Exercice 3

Etape1:

On va creer un network name space par `sudo ip netns add mynamespace`

Et on va le vérifier son existence par la commande

```
sudo ip netns list
```

```
sudo ip netns exec mynamespace ip address
```

```
root@osboxes:/home/osboxes# sudo ip netns add mynamespace
root@osboxes:/home/osboxes# sudo ip netns list
mynamespace
root@osboxes:/home/osboxes# sudo ip netns add mynamespace ip address
Cannot create namespace file "/run/netns/mynamespace": File exists
root@osboxes:/home/osboxes#
```

Etape2

On va Créer un nouveau lien réseau connectant deux nouvelles interfaces (veth0 et veth1). Le lien permet aux données de circuler entre elles.(lien virtuelle)

```
sudo ip link add veth0 type veth peer name veth1
```

Pour vérifier l'interface on a utilisé `sudo ip link set veth1 netns mynamespace`

```
root@osboxes:/home/osboxes# sudo ip link add veth0 type veth peer name veth1
root@osboxes:/home/osboxes# sudo ip link set veth1 netns mynamespace
[1]+  Done                  sleep 1000
root@osboxes:/home/osboxes#
```

Étape 3 : Assignez des adresses IP aux deux interfaces du nouveau lien et activez-les.

On va configurer les adresses IP

```
root@osboxes:/home/osboxes# sudo ip addr add 192.168.1.1
Not enough information: "dev" argument is required.
root@osboxes:/home/osboxes# sudo ip addr add 192.168.1.1 dev veth0
root@osboxes:/home/osboxes# sudo ip link set veth0 up
root@osboxes:/home/osboxes# sudo ip netns exec mynamespace ip addr add 192.168.1.2/24 dev veth1
root@osboxes:/home/osboxes# sudo ip netns exec mynamespace ip link set veth1 up
root@osboxes:/home/osboxes#
```

Étape 4 : Testez la connectivité réseau sur le nouveau lien (veth0 et veth1).

Les cgroups:

Objectif : Limiter et mesurer l'utilisation des ressources (CPU, mémoire, etc.) à l'aide des groupes de contrôle (cgroups). Les cgroups permettent de limiter, mesurer et contrôler l'utilisation des ressources. Les ressources courantes incluent le CPU, la mémoire et l'I/O.

Exercice : Limitation du CPU

Étape 1: Créer un cgroup avec une limitation du CPU.

```
root@osboxes:/sys/fs# stat -fc %T /sys/fs/cgroup/  
cgroup2fs  
root@osboxes:/sys/fs# sudo mkdir /sys/fs/cgroup/mycgroup  
root@osboxes:/sys/fs# echo "20000 100000" | sudo tee /sys/fs/cgroup/mycgroup/cp  
u.max  
20000 100000  
root@osboxes:/sys/fs#
```

20000 représente le montant du temps CPU en microsecondes et 100000 représente la période en microsecondes

Puisque le cpu quota et la periode represente definissent le taux pour lequel les processus de cgroupes peuvent utiliser le cpu par un exemple un cgroup doit utiliser 20ms du temps cpu quota pour chaque periode 100ms

-On alancer un test par :`sudo cgexec -g cpu:mycgroup stress --cpu 1` puis on a surveiller avec Top et on remarque que le cpu ne depasse pas 30%

Exercice: Limitation de la Mémoire

`sudo cgexec -g memory:mycgroup stress --vm 1 --vm-bytes 80M --timeout 60s` o Cette commande lance le stress test et alloue 80 MB de mémoire (i.e., 83,886,080 bytes) et l'exécute pour 60 secondes. o Capture d'écran : Prenez une capture d'écran montrant le processus en cours d'exécution dans le cgroup.

```
osboxes@osboxes:~$ ^C  
osboxes@osboxes:~$ echo 100M | sudo tee /sys/fs/cgroup/mycgroup/memory.max  
[sudo] password for osboxes:  
100M  
osboxes@osboxes:~$ sudo cgexec -g memory:mycgroup stress --vm 1 --vm-bytes 80M -  
-timeout 60s  
sudo: cgexec: command not found  
osboxes@osboxes:~$ sudo cgexec -g memory:mycgroup stress --vm 1 --vm-bytes 80M -  
-timeout 60s  
sudo: cgexec: command not found  
osboxes@osboxes:~$ cat /sys/fs/cgroup//mycgroup/memory.current  
0  
osboxes@osboxes:~$
```